

Chapter 8 Using ACLs and Attributes to Protect ZFS Files (Solaris ZFS Administration Guide)

This chapter provides information about using access control lists (ACLs) to protect your ZFS files by providing more granular permissions than the standard UNIX permissions.

The following sections are provided in this chapter:

- [New Solaris ACL Model](#)
- [Setting ACLs on ZFS Files](#)
- [Setting and Displaying ACLs on ZFS Files in Verbose Format](#)
- [Setting and Displaying ACLs on ZFS Files in Compact Format](#)
- [Applying Special Attributes to ZFS Files](#)

New Solaris ACL Model

Previous versions of Solaris supported an ACL implementation that was primarily based on the POSIX-draft ACL specification. The POSIX-draft based ACLs are used to protect UFS files and are translated by versions of NFS prior to NFSv4.

With the introduction of NFSv4, a new ACL model fully supports the interoperability that NFSv4 offers between UNIX and non-UNIX clients. The new ACL implementation, as defined in the NFSv4 specification, provides much richer semantics that are based on NT-style ACLs.

The main differences of the new ACL model are as follows:

- Based on the NFSv4 specification and similar to NT-style ACLs.
- Provide much more granular set of access privileges. For more information, see [Table 8–2](#).
- Set and displayed with the `chmod` and `ls` commands rather than the `setfacl` and `getfacl` commands.
- Provide richer inheritance semantics for designating how access privileges are applied from directory to subdirectories, and so on. For more information, see [ACL Inheritance](#).

Both ACL models provide more fine-grained access control than is available with the standard file permissions. Much like POSIX-draft ACLs, the new ACLs are composed of multiple Access Control Entries (ACEs).

POSIX-draft style ACLs use a single entry to define what permissions are allowed and what permissions are denied. The new ACL model has two types of ACEs that affect access checking: `ALLOW` and `DENY`. As such, you cannot infer from any single ACE that defines a set of permissions whether or not the permissions that weren't defined in that ACE are allowed or denied.

Translation between NFSv4-style ACLs and POSIX-draft ACLs is as follows:

- If you use any ACL-aware utility, such as the `cp`, `mv`, `tar`, `cpio`, or `rcp` commands, to transfer UFS files with ACLs to a ZFS file system, the POSIX-draft ACLs are translated into the equivalent NFSv4-style ACLs.
- Some NFSv4-style ACLs are translated to POSIX-draft ACLs. You see a message similar to the following if an NFSv4–style ACL isn't translated to a POSIX-draft ACL:

```
# cp -p filea /var/tmp
cp: failed to set acl entries on /var/tmp/filea
```

- If you create a UFS `tar` or `cpio` archive with the preserve ACL option (`tar -p` or `cpio -P`) on a system that runs a current Solaris release, you will lose the ACLs when the archive is extracted on a system that runs a previous Solaris release.

All of the files are extracted with the correct file modes, but the ACL entries are ignored.

- You can use the `ufsrestore` command to restore data into a ZFS file system. If the original data includes POSIX-style ACLs, they are converted to NFSv4-style ACLs.
- If you attempt to set an NFSv4-style ACL on a UFS file, you see a message similar to the following:

```
chmod: ERROR: ACL type's are different
```

- If you attempt to set a POSIX-style ACL on a ZFS file, you will see messages similar to the following:

```
# getfacl filea
File system doesn't support aclent_t style ACL's.
See acl(5) for more information on Solaris ACL support.
```

For information about other limitations with ACLs and backup products, see [Saving ZFS Data With Other Backup Products](#).

Syntax Descriptions for Setting ACLs

Two basic ACL formats are provided as follows:

Syntax for Setting Trivial ACLs

```
chmod [options] A[index]{+|=}owner@ |group@ |everyone@:access-permissions/...[:inheritance-flags]:deny | allow file

chmod [options] A-owner@, group@, everyone@:access-permissions/...[:inheritance-flags]:deny | allow file ...

chmod [options] A[index]- file
```

Syntax for Setting Non-Trivial ACLs

```
chmod [options] A[index]{+|=}user|group:name:access-permissions/...[:inheritance-flags]:deny | allow file

chmod [options] A-user|group:name:access-permissions/...[:inheritance-flags]:deny | allow file ...

chmod [options] A[index]- file

owner@, group@, everyone@
```

Identifies the **ACL-entry-type** for trivial ACL syntax. For a description of **ACL-entry-types**, see [Table 8–1](#).

user or group:**ACL-entry-ID**=username or groupname

Identifies the **ACL-entry-type** for explicit ACL syntax. The user and group **ACL-entry-type** must also contain the **ACL-entry-ID**, **username** or **groupname**. For a description of **ACL-entry-types**, see [Table 8–1](#).

access-permissions/.../

Identifies the access permissions that are granted or denied. For a description of ACL access privileges, see [Table 8–2](#).

inheritance-flags

Identifies an optional list of ACL inheritance flags. For a description of the ACL inheritance flags, see [Table 8–3](#).

deny | allow

Identifies whether the access permissions are granted or denied.

In the following example, the **ACL-entry-ID** value is not relevant.

```
group@:write_data/append_data/execute:deny
```

The following example includes an **ACL-entry-ID** because a specific user (**ACL-entry-type**) is included in the ACL.

```
0:user:gozer:list_directory/read_data/execute:allow
```

When an ACL entry is displayed, it looks similar to the following:

```
2:group@:write_data/append_data/execute:deny
```

The **2** or the **index-ID** designation in this example identifies the ACL entry in the larger ACL, which might have multiple entries for owner, specific UIDs, group, and everyone. You can specify the **index-ID** with the `chmod` command to identify which part of the ACL you want to modify. For example, you can identify index ID 3 as `A3` to the `chmod` command, similar to the following:

```
chmod A3=user:venkman:read_acl:allow filename
```

ACL entry types, which are the ACL representations of owner, group, and other, are described in the following table.

Table 8–1 ACL Entry Types

ACL Entry Type	Description
owner@	Specifies the access granted to the owner of the object.
group@	Specifies the access granted to the owning group of the object.
everyone@	Specifies the access granted to any user or group that does not match any other ACL entry.
user	With a user name, specifies the access granted to an additional user of the object. Must include the ACL-entry-ID , which contains a <i>username</i> or <i>userID</i> . If the value is not a valid numeric UID or <i>username</i> , the ACL entry type is invalid.
group	With a group name, specifies the access granted to an additional group of the object. Must include the ACL-entry-ID , which contains a <i>groupname</i> or <i>groupID</i> . If the value is not a valid numeric GID or <i>groupname</i> , the ACL entry type is invalid.

ACL access privileges are described in the following table.

Table 8–2 ACL Access Privileges

Access Privilege	Compact Access Privilege	Description
add_file	w	Permission to add a new file to a directory.
add_subdirectory	p	On a directory, permission to create a subdirectory.
append_data	p	Placeholder. Not currently implemented.
delete	d	Permission to delete a file.
delete_child	D	Permission to delete a file or directory within a directory.
execute	x	Permission to execute a file or search the contents of a directory.
list_directory	r	Permission to list the contents of a directory.
read_acl	c	Permission to read the ACL (<code>ls</code>).
read_attributes	a	Permission to read basic attributes (non-ACLs) of a file. Think of basic attributes as the <code>stat</code> level attributes. Allowing this access mask bit means the entity can execute <code>ls(1)</code> and <code>stat(2)</code> .
read_data	r	Permission to read the contents of the file.
read_xattr	R	Permission to read the extended attributes of a file or perform a lookup in the file's extended attributes directory.
synchronize	s	Placeholder. Not currently implemented. Permission to create extended attributes or write to the extended attributes directory.
write_xattr	W	Granting this permission to a user means that the user can create an extended attribute directory for a file. The attribute file's permissions control the user's access to the attribute.
write_data	w	Permission to modify or replace the contents of a file.
write_attributes	A	Permission to change the times associated with a file or directory to an arbitrary value.
write_acl	C	Permission to write the ACL or the ability to modify the ACL by using the <code>chmod</code> command. Permission to change the file's owner or group. Or, the ability to execute the <code>chown</code> or <code>chgrp</code> commands on the file.
write_owner	o	Permission to take ownership of a file or permission to change the group ownership of the file to a group of which the user is a member. If you want to change the file or group ownership to an arbitrary user or group, then the <code>PRIV_FILE_CHOWN</code> privilege is required.

ZFS ACL Sets

The following ACL combinations can be applied in an **ACL set** rather than setting individual permissions separately. The following ACL sets are available.

ACL Set Name	Included ACL Permissions
full_set	All permissions
modify_set	all permissions except <code>write_acl</code> and <code>write_owner</code>
read_set	<code>read_data</code> , <code>read_attributes</code> , <code>read_xattr</code> , and <code>read_acl</code>
write_set	<code>write_data</code> , <code>append_data</code> , <code>write_attributes</code> , and <code>write_xattr</code>

These ACL sets are predefined and cannot be modified.

ACL Inheritance

The purpose of using ACL inheritance is so that a newly created file or directory can inherit the ACLs they are intended to inherit, but without disregarding the existing permission bits on the parent directory.

By default, ACLs are not propagated. If you set a non-trivial ACL on a directory, it is not inherited to any subsequent directory. You must specify the inheritance of an ACL on a file or directory.

The optional inheritance flags are described in the following table.

Table 8–3 ACL Inheritance Flags

Inheritance Flag	Compact Inheritance Flag	Description
file_inherit	f	Only inherit the ACL from the parent directory to the directory's files.
dir_inherit	d	Only inherit the ACL from the parent directory to the directory's subdirectories.
inherit_only	i	Inherit the ACL from the parent directory but applies only to newly created files or subdirectories and not the directory itself. This flag requires the <code>file_inherit</code> flag, the <code>dir_inherit</code> flag, or both, to indicate what to inherit.
no_propagate	n	Only inherit the ACL from the parent directory to the first-level contents of the directory, not the second-level or subsequent contents. This flag requires the <code>file_inherit</code> flag, the <code>dir_inherit</code> flag, or both, to indicate what to inherit.
-	N/A	No permission granted.

Inheritance Flag	Compact Inheritance Flag	Description
Currently, the following flags are only applicable to a CIFS client or server.		
successful_access	S	Indicates whether an alarm or audit record should be initiated upon a successful access. This flag is used with audit or alarm ACE types.
failed_access	F	Indicates whether an alarm or audit record should be initiated when an access fails. This flag is used with audit or alarm ACE types.
inherited	I	Indicates that an ACE was inherited.

In addition, you can set a default ACL inheritance policy on the file system that is more strict or less strict by using the `aclinherit` file system property. For more information, see the next section.

ACL Property Modes

The ZFS file system includes two property modes related to ACLs:

- `aclinherit` – This property determines the behavior of ACL inheritance. Values include the following:
 - `discard` – For new objects, no ACL entries are inherited when a file or directory is created. The ACL on the file or directory is equal to the permission mode of the file or directory.
 - `noallow` – For new objects, only inheritable ACL entries that have an access type of `deny` are inherited.
 - `restricted` – For new objects, the `write_owner` and `write_acl` permissions are removed when an ACL entry is inherited.
 - `passthrough` – When property value is set to `passthrough`, files are created with a mode determined by the inheritable ACEs. If no inheritable ACEs exist that affect the mode, then the mode is set in accordance to the requested mode from the application.
 - `passthrough-x` – Has the same semantics as `passthrough`, except that when `passthrough-x` is enabled, files are created with the `execute (x)` permission, but only if `execute` permission is set in the file creation mode and in an inheritable ACE that affects the mode.

The default mode for the `aclinherit` is `restricted`.

- `aclmode` – This property modifies ACL behavior when a file is initially created or whenever a file or directory's mode is modified by the `chmod` command. Values include the following:
 - `discard` – All ACL entries are removed except for the entries needed to define the mode of the file or directory.
 - `groupmask` – User or group ACL permissions are reduced so that they are no greater than the group permission bits, unless it is a user entry that has the same UID as the owner of the file or directory. Then, the ACL permissions are reduced so that they are no greater than owner permission bits.
 - `passthrough` – During a `chmod` operation, ACEs other than `owner@`, `group@`, or `everyone@` are not modified in any way. ACEs with `owner@`, `group@`, or `everyone@` are disabled to set the file mode as requested by the `chmod` operation.

The default mode for the `aclmode` property is `groupmask`.

Setting ACLs on ZFS Files

As implemented with ZFS, ACLs are composed of an array of ACL entries. ZFS provides a **pure** ACL model, where all files have an ACL. Typically, the ACL is **trivial** in that it only represents the traditional UNIX `owner/group/other` entries.

ZFS files still have permission bits and a mode, but these values are more of a cache of what the ACL represents. As such, if you change the permissions of the file, the file's ACL is updated accordingly. In addition, if you remove a non-trivial ACL that granted a user access to a file or directory, that user could still have access to the file or directory because of the file or directory's permission bits that grant access to group or everyone. All access control decisions are governed by the permissions represented in a file or directory's ACL.

The primary rules of ACL access on a ZFS file are as follows:

- ZFS processes ACL entries in the order they are listed in the ACL, from the top down.
- Only ACL entries that have a “who” that matches the requester of the access are processed.
- Once an allow permission has been granted, it cannot be denied by a subsequent ACL deny entry in the same ACL permission set.
- The owner of the file is granted the `write_acl` permission unconditionally, even if the permission is explicitly denied. Otherwise, any permission left unspecified is denied.

In the cases of deny permissions or when an access permission is missing, the privilege subsystem determines what access request is granted for the owner of the file or for superuser. This mechanism prevents owners of files from getting locked out of their files and enables superuser to modify files for recovery purposes.

If you set a non-trivial ACL on a directory, the ACL is not automatically inherited by the directory's children. If you set an non-trivial ACL and you want it inherited to the directory's children, you have to use the ACL inheritance flags. For more information, see [Table 8–3](#) and [Setting ACL Inheritance on ZFS Files in Verbose Format](#).

When you create a new file and depending on the `umask` value, a default trivial ACL, similar to the following, is applied:

```
$ ls -v file.1
-rw-r--r--  1 root    root      206663 Aug 31 11:53 file.1
0:owner@:execute:deny
1:owner@:read_data/write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:allow
2:group@:write_data/append_data/execute:deny
3:group@:read_data:allow
4:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
5:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
  :allow
```

Note that each user category (owner@, group@, everyone@) in this example has two ACL entries. One entry for deny permissions, and one entry is for allow permissions.

A description of this file ACL is as follows:

0:owner@

The owner is denied execute permission to the file (execute:deny).

1:owner@

The owner can read and modify the contents of the file (read_data/write_data/append_data). The owner can also modify the file's attributes such as timestamps, extended attributes, and ACLs (write_xattr/write_attributes /write_acl). In addition, the owner can modify the ownership of the file (write_owner:allow)

2:group@

The group is denied modify and execute permissions to the file (write_data/append_data/execute:deny).

3:group@

The group is granted read permissions to the file (read_data:allow).

4:everyone@

Everyone who is not user or group is denied permission to execute or modify the contents of the file and to modify any attributes of the file (write_data/append_data/write_xattr/execute/write_attributes/write_acl/write_owner:deny).

5:everyone@

Everyone who is not user or group is granted read permissions to the file, and the file's attributes (read_data/read_xattr/read_attributes/read_acl/synchronize:allow). The synchronize access permission is not currently implemented.

When a new directory is created and depending on the umask value, a default directory ACL is similar to the following:

```
$ ls -dv dir.1
drwxr-xr-x  2 root    root      2 Aug 31 11:54 dir.1
0:owner@::deny
1:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
2:group@:add_file/write_data/add_subdirectory/append_data:deny
3:group@:list_directory/read_data/execute:allow
4:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
5:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

A description of this directory ACL is as follows:

0:owner@

The owner deny list is empty for the directory (::deny).

1:owner@

The owner can read and modify the directory contents (list_directory/read_data/add_file/write_data/add_subdirectory/append_data), search the contents (execute), and modify the file's attributes such as timestamps, extended attributes, and ACLs (write_xattr/write_attributes/write_acl). In addition, the owner can modify the ownership of the directory (write_owner:allow).

2:group@

The group cannot add to or modify the directory contents (add_file/write_data/add_subdirectory/append_data:deny).

3:group@

The group can list and read the directory contents. In addition, the group has execute permission to search the directory contents (list_directory/read_data/execute:allow).

4:everyone@

Everyone who is not user or group is denied permission to add to or modify the contents of the directory (add_file/write_data/

add_subdirectory/append_data). In addition, the permission to modify any attributes of the directory is denied. (write_xattr/write_attributes/write_acl/write_owner:deny).

5:everyone@

Everyone who is not user or group is granted read and execute permissions to the directory contents and the directory's attributes (list_directory/read_data/read_xattr/execute/read_attributes/read_acl/synchronize:allow). The synchronize access permission is not currently implemented.

Setting and Displaying ACLs on ZFS Files in Verbose Format

You can use the `chmod` command to modify ACLs on ZFS files. The following `chmod` syntax for modifying ACLs uses **acl-specification** to identify the format of the ACL. For a description of **acl-specification**, see [Syntax Descriptions for Setting ACLs](#).

- Adding ACL entries
 - Adding an ACL entry for a user

```
% chmod A+acl-specification filename
```

- Adding an ACL entry by *index-ID*

```
% chmod Aindex-ID+acl-specification filename
```

This syntax inserts the new ACL entry at the specified *index-ID* location.

- Replacing an ACL entry

```
% chmod A=acl-specification filename
```

```
% chmod Aindex-ID=acl-specification filename
```

- Removing ACL entries
 - Removing an ACL entry by *index-ID*

```
% chmod Aindex-ID- filename
```

- Removing an ACL entry by user

```
% chmod A-acl-specification filename
```

- Removing all non-trivial ACEs from a file

```
% chmod A- filename
```

Verbose ACL information is displayed by using the `ls -v` command. For example:

```
# ls -v file.1
-rw-r--r--  1 root    root      206663 Aug 31 11:53 file.1
0:owner@:execute:deny
1:owner@:read_data/write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:allow
2:group@:write_data/append_data/execute:deny
3:group@:read_data:allow
4:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
5:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
  :allow
```

For information about using the compact ACL format, see [Setting and Displaying ACLs on ZFS Files in Compact Format](#).

Example 8–1 Modifying Trivial ACLs on ZFS Files

This section provides examples of setting and displaying trivial ACLs.

In the following example, a trivial ACL exists on `file.1`:


```
# ls -v file.1
-rw-r--r--  1 root    root      206663 Aug 31 11:53 file.1
0:owner@:execute:deny
1:owner@:read_data/write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:allow
2:group@:write_data/append_data/execute:deny
3:group@:read_data:allow
4:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
5:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
  :allow
```

In the following example, write_data permissions are granted for group@.

```
# chmod A2=group@:append_data/execute:deny file.1
# chmod A3=group@:read_data/write_data:allow file.1
# ls -v file.1
-rw-rw-r--  1 root    root      206663 Aug 31 11:53 file.1
0:owner@:execute:deny
1:owner@:read_data/write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:allow
2:group@:append_data/execute:deny
3:group@:read_data/write_data:allow
4:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
5:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
  :allow
```

In the following example, permissions on file.1 are set back to 644.

```
# chmod 644 file.1
# ls -v file.1
-rw-r--r--  1 root    root      206663 Aug 31 11:53 file.1
0:owner@:execute:deny
1:owner@:read_data/write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:allow
2:group@:write_data/append_data/execute:deny
3:group@:read_data:allow
4:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
5:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
  :allow
```

Example 8–2 Setting Non-Trivial ACLs on ZFS Files

This section provides examples of setting and displaying non-trivial ACLs.

In the following example, read_data/execute permissions are added for the user gozer on the test.dir directory.

```
# chmod A+user:gozer:read_data/execute:allow test.dir
# ls -dv test.dir
drwxr-xr-x+  2 root    root      2 Aug 31 12:02 test.dir
0:user:gozer:list_directory/read_data/execute:allow
1:owner@::deny
2:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
3:group@:add_file/write_data/add_subdirectory/append_data:deny
4:group@:list_directory/read_data/execute:allow
5:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
6:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

In the following example, read_data/execute permissions are removed for user gozer.

```
# chmod A0- test.dir
# ls -dv test.dir
0:owner@::deny
1:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
2:group@:add_file/write_data/add_subdirectory/append_data:deny
3:group@:list_directory/read_data/execute:allow
4:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
5:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

Example 8–3 ACL Interaction With Permissions on ZFS Files

These ACL examples illustrate the interaction between setting ACLs and then changing the file or directory's permission bits.

In the following example, a trivial ACL exists on file.2:

```
# ls -v file.2
-rw-r--r--  1 root    root      2836 Aug 31 12:06 file.2
0:owner@:execute:deny
1:owner@:read_data/write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:allow
2:group@:write_data/append_data/execute:deny
3:group@:read_data:allow
4:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
5:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
  :allow
```

In the following example, ACL allow permissions are removed from everyone@.

```
# chmod A5- file.2
# ls -v file.2
-rw-r-----+ 1 root    root      2836 Aug 31 12:06 file.2
0:owner@:execute:deny
1:owner@:read_data/write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:allow
2:group@:write_data/append_data/execute:deny
3:group@:read_data:allow
4:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
```

In this output, the file's permission bits are reset from 644 to 640. Read permissions for everyone@ have been effectively removed from the file's permissions bits when the ACL allow permissions are removed for everyone@.

In the following example, the existing ACL is replaced with read_data/write_data permissions for everyone@.

```
# chmod A=everyone@:read_data/write_data:allow file.3
# ls -v file.3
-rw-rw-rw-+ 1 root    root      2455 Aug 31 12:08 file.3
0:everyone@:read_data/write_data:allow
```

In this output, the chmod syntax effectively replaces the existing ACL with read_data/write_data:allow permissions to read/write permissions for owner, group, and everyone@. In this model, everyone@ specifies access to any user or group. Since no owner@ or group@ ACL entry exists to override the permissions for owner and group, the permission bits are set to 666.

In the following example, the existing ACL is replaced with read permissions for user gozer.

```
# chmod A=user:gozer:read_data:allow file.3
# ls -v file.3
-----+ 1 root    root      2455 Aug 31 12:08 file.3
0:user:gozer:read_data:allow
```

In this output, the file permissions are computed to be 000 because no ACL entries exist for owner@, group@, or everyone@, which represent the traditional permission components of a file. The owner of the file can resolve this problem by resetting the permissions (and the ACL) as follows:

```
# chmod 655 file.3
# ls -v file.3
-rw-r-xr-x+ 1 root    root      2455 Aug 31 12:08 file.3
0:user:gozer::deny
1:user:gozer:read_data:allow
2:owner@:execute:deny
3:owner@:read_data/write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:allow
4:group@:write_data/append_data:deny
5:group@:read_data/execute:allow
6:everyone@:write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:deny
7:everyone@:read_data/read_xattr/execute/read_attributes/read_acl
  /synchronize:allow
```

Example 8–4 Restoring Trivial ACLs on ZFS Files

You can use the chmod command to remove all non-trivial ACLs on a file or directory.

In the following example, two non-trivial ACEs exist on test5.dir.

```
# ls -dv test5.dir
```



```
drwxr-xr-x+  2 root      root          2 Aug 31 12:11 test5.dir
0:user:lp:read_data:file_inherit:deny
1:user:gozer:read_data:file_inherit:deny
2:owner@::deny
3:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
4:group@:add_file/write_data/add_subdirectory/append_data:deny
5:group@:list_directory/read_data/execute:allow
6:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
7:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

In the following example, the non-trivial ACLs for users `gozer` and `lp` are removed. The remaining ACL contains the six default values for `owner@`, `group@`, and `everyone@`.

```
# chmod A- test5.dir
# ls -dv test5.dir
drwxr-xr-x  2 root      root          2 Aug 31 12:11 test5.dir
0:owner@::deny
1:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
2:group@:add_file/write_data/add_subdirectory/append_data:deny
3:group@:list_directory/read_data/execute:allow
4:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
5:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

Example 8–5 Applying an ACL Set to ZFS Files

ACL sets are available so that you do not have to apply ACL permissions separately. For a description of ACL sets, see [ZFS ACL Pass Through Inheritance for Execute Permission](#).

For example, you can apply the `write_set` as follows:

```
# chmod A+user:otto:read_set:allow file.1
# ls -v file.1
-r--r--r--+  1 root      root          206674 Aug  1 13:18 file.1
0:user:otto:read_data/read_xattr/read_attributes/read_acl:allow
1:owner@:write_data/append_data/execute:deny
2:owner@:read_data/write_xattr/write_attributes/write_acl/write_owner
  :allow
3:group@:write_data/append_data/execute:deny
4:group@:read_data:allow
5:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
6:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
  :allow
```

You can apply the `write_set` and `read_set` as follows:

```
# chmod A+user:otto:read_set/write_set:allow file.2
# ls -v file.2
-r--r--r--+  1 root      root          206674 Aug  1 13:19 file.2
0:user:otto:read_data/write_data/append_data/read_xattr/write_xattr
  /read_attributes/write_attributes/read_acl:allow
1:owner@:write_data/append_data/execute:deny
2:owner@:read_data/write_xattr/write_attributes/write_acl/write_owner
  :allow
3:group@:write_data/append_data/execute:deny
4:group@:read_data:allow
5:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
6:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
  :allow
```

Setting ACL Inheritance on ZFS Files in Verbose Format

You can determine how ACLs are inherited or not inherited on files and directories. By default, ACLs are not propagated. If you set a non-trivial ACL on a directory, the ACL is not inherited by any subsequent directory. You must specify the inheritance of an ACL on a file or directory.

In addition, two ACL properties are provided that can be set globally on file systems: `aclinherit` and `aclmode`. By default, *aclinherit* is set to `restricted` and *aclmode* is set to `groupmask`.

For more information, see [ACL Inheritance](#).

Example 8–6 Granting Default ACL Inheritance

By default, ACLs are not propagated through a directory structure.

In the following example, a non-trivial ACE of read_data/write_data/execute is applied for user gozer on test.dir.

```
# chmod A+user:gozer:read_data/write_data/execute:allow test.dir
# ls -dv test.dir
drwxr-xr-x+  2 root      root          2 Aug 31 13:02 test.dir
0:user:gozer:list_directory/read_data/add_file/write_data/execute:allow
1:owner@::deny
2:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
3:group@:add_file/write_data/add_subdirectory/append_data:deny
4:group@:list_directory/read_data/execute:allow
5:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
6:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

If a test.dir subdirectory is created, the ACE for user gozer is not propagated. User gozer would only have access to sub.dir if the permissions on sub.dir granted him access as the file owner, group member, or everyone@.

```
# mkdir test.dir/sub.dir
# ls -dv test.dir/sub.dir
drwxr-xr-x  2 root      root          2 Aug 31 13:26 test.dir/sub.dir
0:owner@::deny
1:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
2:group@:add_file/write_data/add_subdirectory/append_data:deny
3:group@:list_directory/read_data/execute:allow
4:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
5:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

Example 8–7 Granting ACL Inheritance on Files and Directories

This series of examples identify the file and directory ACEs that are applied when the file_inherit flag is set.

In the following example, read_data/write_data permissions are added for files in the test.dir directory for user gozer so that he has read access on any newly created files.

```
# chmod A+user:gozer:read_data/write_data:file_inherit:allow test2.dir
# ls -dv test2.dir
drwxr-xr-x+  2 root      root          2 Aug 31 13:26 test2.dir
0:user:gozer:read_data/write_data:file_inherit:allow
1:owner@::deny
2:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
3:group@:add_file/write_data/add_subdirectory/append_data:deny
4:group@:list_directory/read_data/execute:allow
5:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
6:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

In the following example, user gozer's permissions are applied on the newly created test2.dir/file.2 file. The ACL inheritance granted, read_data:file_inherit:allow, means user gozer can read the contents of any newly created file.

```
# touch test2.dir/file.2
# ls -v test2.dir/file.2
-rw-r--r--+  1 root      root          0 Aug 31 13:27 test2.dir/file.2
0:user:gozer:write_data:deny
1:user:gozer:read_data/write_data:allow
2:owner@:execute:deny
3:owner@:read_data/write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:allow
4:group@:write_data/append_data/execute:deny
5:group@:read_data:allow
6:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
7:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
  :allow
```

Because the *aclmode* for this file is set to the default mode, groupmask, user gozer does not have write_data permission on file.2 because the group permission of the file does not allow it.

Note the `inherit_only` permission, which is applied when the `file_inherit` or `dir_inherit` flags are set, is used to propagate the ACL through the directory structure. As such, user `gozer` is only granted or denied permission from `everyone@` permissions unless he is the owner of the file or a member of the owning group of the file. For example:

```
# mkdir test2.dir/subdir.2
# ls -dv test2.dir/subdir.2
drwxr-xr-x+  2 root    root          2 Aug 31 13:28 test2.dir/subdir.2
0:user:gozer:list_directory/read_data/add_file/write_data:file_inherit
  /inherit_only:allow
1:owner@::deny
2:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
3:group@:add_file/write_data/add_subdirectory/append_data:deny
4:group@:list_directory/read_data/execute:allow
5:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
6:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

The following series of examples identify the file and directory ACLs that are applied when both the `file_inherit` and `dir_inherit` flags are set. In the following example, user `gozer` is granted read, write, and execute permissions that are inherited for newly created files and directories.

```
# chmod A+user:gozer:read_data/write_data/execute:file_inherit/dir_inherit:allow
test3.dir
# ls -dv test3.dir
drwxr-xr-x+  2 root    root          2 Aug 31 13:29 test3.dir
0:user:gozer:list_directory/read_data/add_file/write_data/execute
  :file_inherit/dir_inherit:allow
1:owner@::deny
2:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
3:group@:add_file/write_data/add_subdirectory/append_data:deny
4:group@:list_directory/read_data/execute:allow
5:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
6:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

The inherited text in the output below is an informational message that indicates that the ACE is inherited.

```
# touch test3.dir/file.3
# ls -v test3.dir/file.3
-rw-r--r--+  1 root    root          0 Jun 20 14:44 test3.dir/file.3
0:user:gozer:write_data/execute:deny
1:user:gozer:read_data/write_data/execute:inherited:allow
2:owner@:execute:deny
3:owner@:read_data/write_data/append_data/write_xattr/write_attributes
  /write_acl/write_owner:allow
4:group@:write_data/append_data/execute:deny
5:group@:read_data:allow
6:everyone@:write_data/append_data/write_xattr/execute/write_attributes
  /write_acl/write_owner:deny
7:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
  :allow
```

```
# mkdir test3.dir/subdir.1
# ls -dv test3.dir/subdir.1
drwxr-xr-x+  2 root    root          2 Jun 20 14:45 test3.dir/subdir.1
0:user:gozer:list_directory/read_data/add_file/write_data/execute
  :file_inherit/dir_inherit/inherit_only/inherited:allow
1:user:gozer:add_file/write_data:deny
2:user:gozer:list_directory/read_data/add_file/write_data/execute
  :inherited:allow
3:owner@::deny
4:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
  /append_data/write_xattr/execute/write_attributes/write_acl
  /write_owner:allow
5:group@:add_file/write_data/add_subdirectory/append_data:deny
6:group@:list_directory/read_data/execute:allow
7:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
  /write_attributes/write_acl/write_owner:deny
8:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
  /read_acl/synchronize:allow
```

In these examples, because the permission bits of the parent directory for `group@` and `everyone@` deny write and execute permissions, user `gozer` is denied write and execute permissions. The default *aclmode* property is restricted, which means that `write_data` and `execute` permissions are not inherited.

In the following example, user `gozer` is granted read, write, and execute permissions that are inherited for newly created files, but are not propagated to subsequent contents of the directory.

```
# chmod A+user:gozer:read_data/write_data/execute:file_inherit/no_propagate:allow
test4.dir
# ls -dv test4.dir
drwxr-xr-x+  2 root      root          2 Aug 31 13:34 test4.dir
0:user:gozer:list_directory/read_data/add_file/write_data/execute
:file_inherit/no_propagate:allow
1:owner@::deny
2:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
/append_data/write_xattr/execute/write_attributes/write_acl
/write_owner:allow
3:group@:add_file/write_data/add_subdirectory/append_data:deny
4:group@:list_directory/read_data/execute:allow
5:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
/write_attributes/write_acl/write_owner:deny
6:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
/read_acl/synchronize:allow
```

As the following example illustrates, when a new subdirectory is created, user gozer's read_data/write_data/execute permission for files are not propagated to the new sub4.dir directory.

```
mkdir test4.dir/sub4.dir
# ls -dv test4.dir/sub4.dir
drwxr-xr-x  2 root      root          2 Aug 31 13:35 test4.dir/sub4.dir
0:owner@::deny
1:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
/append_data/write_xattr/execute/write_attributes/write_acl
/write_owner:allow
2:group@:add_file/write_data/add_subdirectory/append_data:deny
3:group@:list_directory/read_data/execute:allow
4:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
/write_attributes/write_acl/write_owner:deny
5:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
/read_acl/synchronize:allow
```

As the following example illustrates, gozer's read_data/write_data/execute permission for files is propagated to the newly created file.

```
# touch test4.dir/file.4
# ls -v test4.dir/file.4
-rw-r--r--+  1 root      root          0 Jun 20 15:15 test4.dir/file.4
0:user:gozer:write_data/execute:deny
1:user:gozer:read_data/write_data/execute:inherited:allow
2:owner@:execute:deny
3:owner@:read_data/write_data/append_data/write_xattr/write_attributes
/write_acl/write_owner:allow
4:group@:write_data/append_data/execute:deny
5:group@:read_data:allow
6:everyone@:write_data/append_data/write_xattr/execute/write_attributes
/write_acl/write_owner:deny
7:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
:allow
```

Example 8–8 ACL Inheritance With ACL Mode Set to Pass Through

If the `aclmode` property on the `tank/cindys` file system is set to `passthrough`, then user gozer would inherit the ACL applied on `test4.dir` for the newly created `file.4` as follows:

```
# zfs set aclmode=passthrough tank/cindys
# touch test4.dir/file.4
# ls -v test4.dir/file.4
-rw-r--r--+  1 root      root          0 Jun 20 15:19 test4.dir/file.4
0:user:gozer:write_data/execute:deny
1:user:gozer:read_data/write_data/execute:inherited:allow
2:owner@:execute:deny
3:owner@:read_data/write_data/append_data/write_xattr/write_attributes
/write_acl/write_owner:allow
4:group@:write_data/append_data/execute:deny
5:group@:read_data:allow
6:everyone@:write_data/append_data/write_xattr/execute/write_attributes
/write_acl/write_owner:deny
7:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
:allow
```

This output illustrates that the `read_data/write_data/execute:allow:file_inherit/dir_inherit` ACL that was set on the parent directory, `test4.dir`, is passed through to user gozer.

Example 8–9 ACL Inheritance With ACL Mode Set to Discard

If the `aclmode` property on a file system is set to `discard`, then ACLs can potentially be discarded when the permission bits on a directory

change. For example:

```
# zfs set aclmode=discard tank/cindys
# chmod A+user:gozer:read_data/write_data/execute:dir_inherit:allow test5.dir
# ls -dv test5.dir
drwxr-xr-x+  2 root      root          2 Aug 31 13:40 test5.dir
0:user:gozer:list_directory/read_data/add_file/write_data/execute
:dir_inherit:allow
1:owner@::deny
2:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
/append_data/write_xattr/execute/write_attributes/write_acl
/write_owner:allow
3:group@:add_file/write_data/add_subdirectory/append_data:deny
4:group@:list_directory/read_data/execute:allow
5:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
/write_attributes/write_acl/write_owner:deny
6:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
/read_acl/synchronize:allow
```

If, at a later time, you decide to tighten the permission bits on a directory, the non-trivial ACL is discarded. For example:

```
# chmod 744 test5.dir
# ls -dv test5.dir
drwxr--r--  2 root      root          2 Aug 31 13:40 test5.dir
0:owner@::deny
1:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
/append_data/write_xattr/execute/write_attributes/write_acl
/write_owner:allow
2:group@:add_file/write_data/add_subdirectory/append_data/execute:deny
3:group@:list_directory/read_data:allow
4:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
/execute/write_attributes/write_acl/write_owner:deny
5:everyone@:list_directory/read_data/read_xattr/read_attributes/read_acl
/synchronize:allow
```

Example 8–10 ACL Inheritance With ACL Inherit Mode Set to Noallow

In the following example, two non-trivial ACLs with file inheritance are set. One ACL allows `read_data` permission, and one ACL denies `read_data` permission. This example also illustrates how you can specify two ACEs in the same `chmod` command.

```
# zfs set aclinherit=noallow tank/cindys
# chmod A+user:gozer:read_data:file_inherit:deny,user:lp:read_data:file_inherit:allow test6.dir
# ls -dv test6.dir
drwxr-xr-x+  2 root      root          2 Aug 31 13:43 test6.dir
0:user:gozer:read_data:file_inherit:deny
1:user:lp:read_data:file_inherit:allow
2:owner@::deny
3:owner@:list_directory/read_data/add_file/write_data/add_subdirectory
/append_data/write_xattr/execute/write_attributes/write_acl
/write_owner:allow
4:group@:add_file/write_data/add_subdirectory/append_data:deny
5:group@:list_directory/read_data/execute:allow
6:everyone@:add_file/write_data/add_subdirectory/append_data/write_xattr
/write_attributes/write_acl/write_owner:deny
7:everyone@:list_directory/read_data/read_xattr/execute/read_attributes
/read_acl/synchronize:allow
```

As the following example shows, when a new file is created, the ACL that allows `read_data` permission is discarded.

```
# touch test6.dir/file.6
# ls -v test6.dir/file.6
-rw-r--r--  1 root      root          0 Aug 31 13:44 test6.dir/file.6
0:owner@:execute:deny
1:owner@:read_data/write_data/append_data/write_xattr/write_attributes
/write_acl/write_owner:allow
2:group@:write_data/append_data/execute:deny
3:group@:read_data:allow
4:everyone@:write_data/append_data/write_xattr/execute/write_attributes
/write_acl/write_owner:deny
5:everyone@:read_data/read_xattr/read_attributes/read_acl/synchronize
:allow
```

Setting and Displaying ACLs on ZFS Files in Compact Format

You can set and display permissions on ZFS files in a compact format that uses 14 unique letters to represent the permissions. The letters that represent the compact permissions are listed in [Table 8–2](#) and [Table 8–3](#).

You can display compact ACL listings for files and directories by using the `ls -v` command. For example:

```
# ls -V file.1
-rw-r--r--  1 root      root      206674 Jul 29 10:27 file.1
  owner@:--x-----:-----:deny
  owner@:rw-p---A-W-Co-:-----:allow
  group@:-wxp-----:-----:deny
  group@:r-----:-----:allow
  everyone@:-wxp---A-W-Co-:-----:deny
  everyone@:r-----a-R-c--s:-----:allow
```

The compact ACL output is described as follows:

owner@

The owner is denied execute permissions to the file (x=execute).

owner@

The owner can read and modify the contents of the file (rw=read_data/write_data), (p=append_data). The owner can also modify the file's attributes such as timestamps, extended attributes, and ACLs (A=write_xattr, W=write_attributes, C=write_acl). In addition, the owner can modify the ownership of the file (o=write_owner).

group@

The group is denied modify and execute permissions to the file (write_data, p=append_data, and x=execute).

group@

The group is granted read permissions to the file (r=read_data).

everyone@

Everyone who is not user or group is denied permission to execute or modify the contents of the file, and to modify any attributes of the file (w=write_data, x=execute, p=append_data, A=write_xattr, W=write_attributes, C=write_acl, and o=write_owner).

everyone@

Everyone who is not user or group is granted read permissions to the file and the file's attributes (r=read_data, a=append_data, R=read_xattr, c=read_acl, and s=synchronize). The synchronize access permission is not currently implemented.

Compact ACL format provides the following advantages over verbose ACL format:

- Permissions can be specified as positional arguments to the `chmod` command.
- The hyphen (-) characters, which identify no permissions, can be removed and only the required letters need to be specified.
- Both permissions and inheritance flags are set in the same fashion.

For information about using the verbose ACL format, see [Setting and Displaying ACLs on ZFS Files in Verbose Format](#).

Example 8–11 Setting and Displaying ACLs in Compact Format

In the following example, a trivial ACL exists on `file.1`:

```
# ls -V file.1
-rw-r--r--  1 root      root      206674 Jul 29 10:27 file.1
  owner@:--x-----:-----:deny
  owner@:rw-p---A-W-Co-:-----:allow
  group@:-wxp-----:-----:deny
  group@:r-----:-----:allow
  everyone@:-wxp---A-W-Co-:-----:deny
  everyone@:r-----a-R-c--s:-----:allow
```

In this example, read_data/execute permissions are added for the user `gozer` on `file.1`.

```
# chmod A+user:gozer:rx:allow file.1
# ls -V file.1
-rw-r--r--+  1 root      root      206674 Jul 29 10:27 file.1
  user:gozer:r-x-----:-----:allow
  owner@:--x-----:-----:deny
  owner@:rw-p---A-W-Co-:-----:allow
  group@:-wxp-----:-----:deny
  group@:r-----:-----:allow
  everyone@:-wxp---A-W-Co-:-----:deny
  everyone@:r-----a-R-c--s:-----:allow
```

Another way to add the same permissions for user `gozer` is to insert a new ACL at a specific position, 4, for example. As such, the existing ACLs at positions 4–6 are pushed down. For example:


```
# chmod A+user:gozer:rx:allow file.1
# ls -V file.1
-rw-r--r--+ 1 root      root      206674 Jul 29 10:27 file.1
  user:gozer:r-x-----:-----:allow
  owner@:--x-----:-----:deny
  owner@:rw-p---A-W-Co-:-----:allow
  group@:-wxp-----:-----:deny
  user:gozer:r-x-----:-----:allow
  group@:r-----:-----:allow
  everyone@:-wxp---A-W-Co-:-----:deny
  everyone@:r-----a-R-c--s:-----:allow
```

In the following example, user gozer is granted read, write, and execute permissions that are inherited for newly created files and directories by using the compact ACL format.

```
# chmod A+user:gozer:rw:fd:allow dir.2
# ls -dV dir.2
drwxr-xr-x+ 2 root      root      2 Jul 29 10:44 dir.2
  user:gozer:rw-----:fd-----:allow
  owner@:-----:-----:deny
  owner@:rwxp---A-W-Co-:-----:allow
  group@:-w-p-----:-----:deny
  group@:r-x-----:-----:allow
  everyone@:-w-p---A-W-Co-:-----:deny
  everyone@:r-x---a-R-c--s:-----:allow
```

You can also cut and paste permissions and inheritance flags from the `ls -v` output into the compact `chmod` format. For example, to duplicate the permissions and inheritance flags on `dir.2` for user gozer to user cindys on `dir.2`, copy and paste the permission and inheritance flags (`rw-----:f-----:allow`) into your `chmod` command. For example:

```
# chmod A+user:cindys:rw-----:fd-----:allow dir.2
# ls -dV dir.2
drwxr-xr-x+ 2 root      root      2 Jul 29 10:44 dir.2
  user:cindys:rw-----:fd-----:allow
  user:gozer:rw-----:fd-----:allow
  owner@:-----:-----:deny
  owner@:rwxp---A-W-Co-:-----:allow
  group@:-w-p-----:-----:deny
  group@:r-x-----:-----:allow
  everyone@:-w-p---A-W-Co-:-----:deny
  everyone@:r-x---a-R-c--s:-----:allow
```

Example 8–12 ACL Inheritance With ACL Inherit Mode Set to Pass Through

A file system that has the `aclinherit` property set to `passthrough` inherits all inheritable ACL entries without any modifications made to the ACL entries when they are inherited. When this property is set to `passthrough`, files are created with a permission mode that is determined by the inheritable ACEs. If no inheritable ACEs exist that affect the permission mode, then the permission mode is set in accordance to the requested mode from the application.

The following examples use compact ACL syntax to show how to inherit permission bits by setting `aclinherit` mode to `passthrough`.

In this example, an ACL is set on `test1.dir` to force inheritance. The syntax creates an `owner@`, `group@`, and `everyone@` ACL entry for newly created files. Newly created directories inherit an `@owner`, `group@`, and `everyone@` ACL entry. Additionally, directories inherit 6 other ACEs that propagate the ACEs to newly created directories and files.

```
# zfs set aclinherit=passthrough tank/cindys
# pwd
/tank/cindys
# mkdir test1.dir

# chmod A=owner@:rwxpcCosRrWaAdD:fd:allow,group@:rwxp:fd:allow,everyone@::fd:allow test1.dir
# ls -Vd test1.dir
drwxrwx---+ 2 root      root      2 Jun 20 15:26 test1.dir
  owner@:rwxpdDaARWcCos:fd-----:allow
  group@:rwxp-----:fd-----:allow
  everyone@:-----:fd-----:allow
```

In this example, a newly created file inherits the ACL that was specified to be inherited to newly created files.

```
# cd test1.dir
# touch file.1
# ls -V file.1
-rwxrwx---+ 1 root      root      0 Jun 23 09:38 file.1
  owner@:rwxpdDaARWcCos:-----I:allow
  group@:rwxp-----:-----I:allow
  everyone@:-----:-----I:allow
```


In this example, a newly created directory inherits both ACEs that control access to this directory as well as ACEs for future propagation to children of the newly created directory.

```
# mkdir subdir.1
# ls -dV subdir.1
drwxrwx---+  2 root      root          2 Jun 23 09:41 subdir.1
      owner@:rwxpdDaARWcCos:fdi---I:allow
      owner@:rwxpdDaARWcCos:-----I:allow
      group@:rwxp-----:fdi---I:allow
      group@:rwxp-----:-----I:allow
      everyone@:-----:fdi---I:allow
      everyone@:-----:-----I:allow
```

The `-di--` and `f-i---` entries are for propagating inheritance and are not considered during access control. In this example, a file is created with a trivial ACL in another directory where inherited ACEs are not present.

```
# cd /tank/cindys
# mkdir test2.dir
# cd test2.dir
# touch file.2
# ls -V file.2
-rw-r--r--  1 root      root          0 Jun 23 09:50 file.2
      owner@:--x-----:-----:deny
      owner@:rw-p---A-W-Co-:-----:allow
      group@:-wxp-----:-----:deny
      group@:r-----:-----:allow
      everyone@:-wxp---A-W-Co-:-----:deny
      everyone@:r-----a-R-c--s:-----:allow
```

Example 8–13 ACL Inheritance With ACL Inherit Mode Set to Pass Through-X

When `aclinherit=passthrough-x` is enabled, files are created with the execute (x) permission for `owner@`, `group@`, or `everyone@`, but only if execute permission is set in the file creation mode and in an inheritable ACE that affects the mode.

The following example shows how to inherit the execute permission by setting `aclinherit` mode to `passthrough-x`.

```
# zfs set aclinherit=passthrough-x tank/cindys
```

The following ACL is set on `/tank/cindys/test1.dir` to provide executable ACL inheritance for files for `owner@`,

```
# chmod A=owner@:rwxpcCosRrWaAdD:fd:allow,group@:rwxp:fd:allow,everyone@::fd:allow test1.dir
# ls -Vd test1.dir
drwxrwx---+  2 root      root          2 Dec  3 15:16 test1.dir
      owner@:rwxpdDaARWcCos:fd-----:allow
      group@:rwxp-----:fd-----:allow
      everyone@:-----:fd-----:allow
```

A file (`file1`) is created with requested permissions `0666`. The resulting permissions are `0660`. The execution permission was not inherited because the creation mode did not request it.

```
# touch test1.dir/file1
# ls -V test1.dir/file1
-rw-rw----+  1 root      root          0 Dec  3 15:17 file.1
      owner@:rw-pdDaARWcCos:-----I:allow
      group@:rw-p-----:-----I:allow
      everyone@:-----:-----I:allow
```

Next, an executable called `t` is generated by using the `cc` compiler in the `testdir` directory.

```
# cc -o t t.c
# ls -V t
-rwxrwx---+  1 root      root          7396 Dec  3 15:19 t
      owner@:rwxpdDaARWcCos:-----I:allow
      group@:rwxp-----:-----I:allow
      everyone@:-----:-----I:allow

# cc -o t t.c
# ls -V t
-rwxrwx---+  1 root      root          7396 Dec  3 15:19 t
      owner@:rwxpdDaARWcCos:-----:allow
      group@:rwxp-----:-----:allow
      everyone@:-----:-----:allow
```

The resulting permissions are `0770` because `cc` requested permissions `0777`, which caused the execute permission to be inherited from the `owner@`,

group@, and everyone@ entries.

Applying Special Attributes to ZFS Files

The following examples show how to apply and display special attributes, such as immutability or read-only access, to ZFS files.

For more information about displaying and applying special attributes, see [ls\(1\)](#) and [chmod\(1\)](#).

Example 8–14 Apply Immutability to a ZFS File

Use the following syntax to make a file immutable:

```
# chmod S+ci file.1
# echo this >>file.1
file.2: cannot create
# rm file.1
rm: file.1: override protection 644 (yes/no)? yes
rm: file.1 not removed: Not owner
```

You can display special attributes on ZFS files by using the following syntax:

```
# ls -l/c file.1
-rw-r--r--  1 root      root      206674 Jul 31 10:52 file.1
              {A-----im--}
```

Use the following syntax to remove file immutability:

```
# chmod S-ci file.1
# ls -l/c file.1
-rw-r--r--+  1 root      root      206674 Jul 29 10:27 file.1
              {A-----m--}
# rm file.1
```

Example 8–15 Apply Read-Only Access to a ZFS File

The following example shows how to apply read-only access to a ZFS file.

```
# chmod S+cR file.2
# echo this >>file.2
file.2: cannot create
```

Example 8–16 Displaying and Removing ZFS File Attributes

You can display all special attributes with the following syntax:

```
# ls -l/v file.3
-r--r--r--  1 root      root      206674 Jul 31 11:30 file.3
              {archive,nohidden,noreadonly,nosystem,noappendonly,nonodump,immutable,
av_modified,noav_quarantined,nonounlink}
# chmod S+cR file.3
# ls -l/v file.3
-r--r--r--  1 root      root      206674 Jul 31 11:30 file.3
              {archive,nohidden,readonly,nosystem,noappendonly,nonodump,immutable,
av_modified,noav_quarantined,nonounlink}
```

Some of these attributes only apply in a CIFS environment.

You can clear all attributes on a file. For example:

```
# chmod S-a file.3
# ls -l/v file.3
-r--r--r--  1 root      root      206674 Jul 31 11:30 file.3
              {noarchive,nohidden,noreadonly,nosystem,noappendonly,nonodump,noimmutable,
noav_modified,noav_quarantined,nonounlink}
```
